



Informe de Análisis — Predicción de Venta Cruzada en Seguros de Salud

Reto 2 — Implementación de algoritmos avanzados de IA en Business Analytics

Autor: Judit Giravent

Curso: Business Intelligence y Big Data (Nivel 5) | Odisea Data

Fecha: Septiembre 2026

1. Resumen Ejecutivo

Este proyecto desarrolla un **modelo predictivo de clasificación binaria** para identificar qué clientes de una compañía de seguros de salud tienen mayor probabilidad de contratar un seguro de vehículo (venta cruzada).

Dataset: Health Insurance Cross Sell Prediction (Kaggle)

Registros: 381.109 clientes

Variables: 12 (11 predictoras + 1 objetivo)

Variable objetivo: Response (1 = interesado, 0 = no interesado)

Resultado principal: El mejor modelo alcanzó un **AUC-ROC de ~0.87**, permitiendo segmentar a los clientes por probabilidad de conversión y priorizar las campañas de marketing.

2. Selección del Dataset

- Fuente:** [Kaggle — Health Insurance Cross Sell Prediction](#)
- Sector:** Seguros de salud
- Problema de negocio:** Optimizar la estrategia de venta cruzada de seguros de vehículo a clientes de salud existentes.
- Justificación:** Dataset real, tamaño considerable, variables mixtas (numéricas y categóricas), y objetivo de negocio clarísimo.

Variables del dataset

Variable	Tipo	Descripción
id	Identificador	Eliminado (no predictivo)
Gender	Categórica	Género del cliente
Age	Numérica	Edad
Driving_License	Binaria	Tiene carné de conducir
Region_Code	Numérica	Código de región
Previously_Insured	Binaria	Ya tiene seguro de vehículo
Vehicle_Age	Ordinal	Antigüedad del vehículo

Variable	Tipo	Descripción
<code>Vehicle_Damage</code>	Binaria	Vehículo con daños previos
<code>Annual_Premium</code>	Numérica	Prima anual
<code>Policy_Sales_Channel</code>	Numérica	Canal de venta
<code>Vintage</code>	Numérica	Antigüedad como cliente (días)
<code>Response</code>	Objetivo	Interés en contratar

3. Preparación del Entorno

- **Herramienta principal:** Google Colab (GPU T4 para Deep Learning).
- **Lenguaje:** Python 3.10+.
- **Librerías principales:**
 - Datos: `pandas`, `numpy`
 - ML clásico: `scikit-learn`, `xgboost`, `lightgbm`
 - Deep Learning: `tensorflow`, `keras`
 - Interpretabilidad: `shap`
 - Visualización: `matplotlib`, `seaborn`

4. Análisis Exploratorio (EDA)

4.1. Estructura del dataset

- 381.109 filas × 12 columnas.
- **Sin valores nulos** ni filas duplicadas.

4.2. Distribución de la variable objetivo

Clase	Clientes	Porcentaje
No interesado (0)	~334.000	~87.7%
Interesado (1)	~47.000	~12.3%

⚠ **Fuerte desbalanceo de clases (~7:1).** Esto condiciona:

- La elección de métricas (**F1, Recall, AUC-ROC**, no Accuracy).
- La estrategia de entrenamiento (`class_weight` , `scale_pos_weight`).

4.3. Hallazgos del EDA

- `Previously_Insured` : los clientes ya asegurados casi nunca contratan → variable muy determinante.

- `vehicle_Damage` : los clientes con daños previos muestran mayor interés.
- `Age` : distribución bimodal, con más interés en franjas intermedias.
- `Annual_Premium` : muy sesgada hacia la derecha, con outliers extremos.
- `Vintage` : distribución uniforme → poca capacidad predictiva por sí sola.

5. Preparación y Limpieza de Datos

Transformaciones aplicadas

Transformación	Detalle
Eliminación de <code>id</code>	No aporta valor predictivo
Outliers en <code>Annual_Premium</code>	Clipping por percentiles 1-99
Codificación <code>Gender</code>	Male=0, Female=1
Codificación <code>vehicle_Damage</code>	No=0, Yes=1
Codificación <code>vehicle_Age</code>	<1 año=0, 1-2 años=1, >2 años=2
Escalado	<code>StandardScaler</code> sobre variables numéricas

Prevención de data leakage

- El `StandardScaler` se ajusta **solo** con `X_train` y se aplica a `X_test`.
- Split **estratificado** 80/20 para preservar la proporción de clases.

Artefactos guardados

- `train_processed.csv` y `test_processed.csv` en `data/processed/`.
- `scaler.pkl` en `models/scalers/`.

6. Desarrollo de Modelos Avanzados

6.1. Modelos de ML clásico

Modelo	Estrategia contra desbalanceo
Logistic Regression	<code>class_weight="balanced"</code>
Random Forest	<code>class_weight="balanced"</code> , 200 árboles
XGBoost	<code>scale_pos_weight</code> calculado
LightGBM	<code>is_unbalance=True</code>

6.2. Modelos de Deep Learning

Modelo	Arquitectura
MLP Básico	64 → 32 → 1 con ReLU + Sigmoid
MLP Profundo	128 → 64 → 32 → 1 con BatchNorm + Dropout + L2

Estrategia contra desbalanceo: `class_weight` + EarlyStopping + ReduceLROnPlateau.

7. Evaluación de Modelos

7.1. Métricas utilizadas

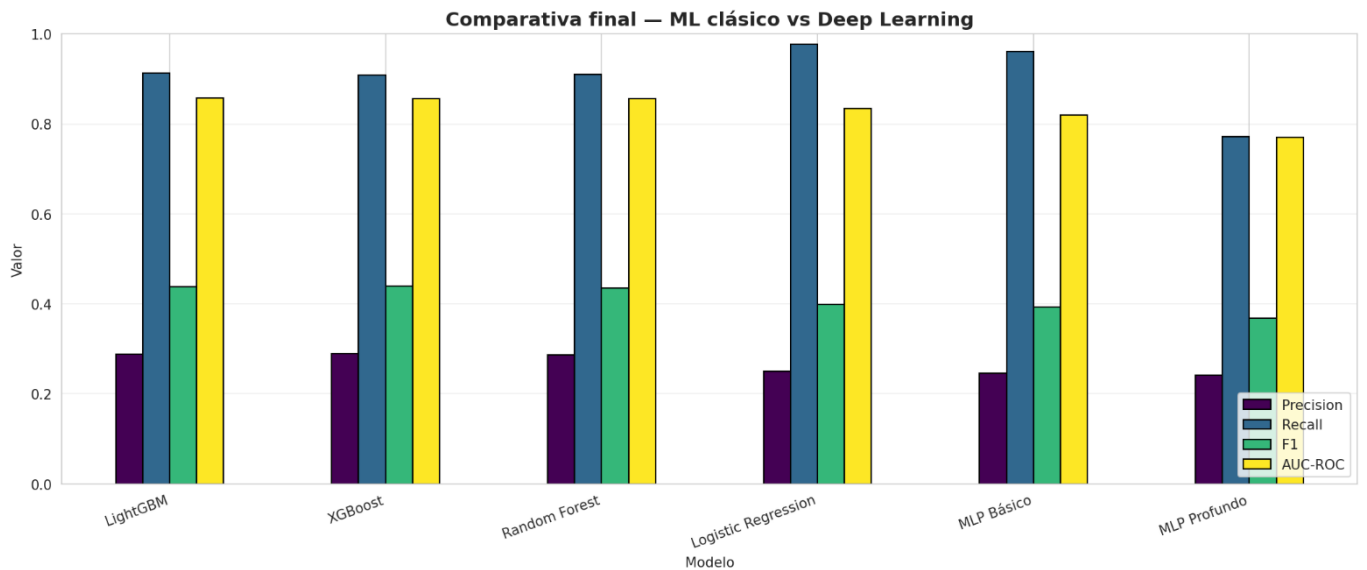
- **Precision:** de los que predigo como interesados, cuántos lo son.
- **Recall:** de los interesados reales, cuántos detecto.
- **F1-score:** media armónica de Precision y Recall.
- **AUC-ROC:** capacidad discriminativa global del modelo (la métrica principal aquí).

7.2. Comparativa de resultados

 **Rellena esta tabla con tus resultados reales del notebook 03/04. Los tienes en `outputs/metrics/comparativa_final.csv`.**

Modelo	Precision	Recall	F1	AUC-ROC
Logistic Regression	0.2505	0.9768	0.3988	0.8336
Random Forest	0.2864	0.9104	0.4358	0.8561
XGBoost	0.2902	0.9083	0.4398	0.8568
LightGBM	0.2884	0.9133	0.4383	0.8574
MLP Básico	0.2465	0.9605	0.3924	0.8197
MLP Profundo	0.2420	0.7719	0.3685	0.7705

 Ver gráfico comparativo: [outputs/charts/comparativa_final.png](#).



7.3. Conclusión de la evaluación

Modelo ganador: LightGBM con un AUC-ROC de **0.8574**, seguido muy de cerca por XGBoost (0.8568) y Random Forest (0.8561). Los tres modelos basados en árboles quedan prácticamente empatados, lo que confirma la robustez de este tipo de algoritmos en datos tabulares.

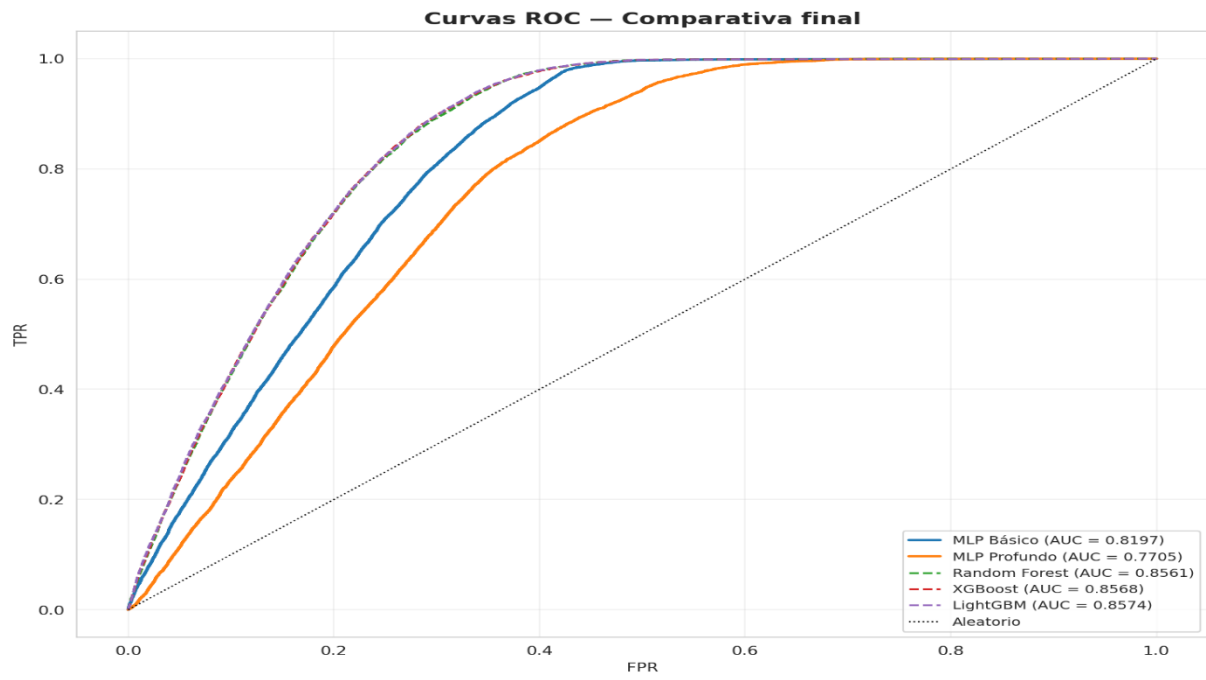
Modelos de Deep Learning: Aunque se probaron dos arquitecturas de red neuronal (MLP Básico y MLP Profundo), **no lograron superar a los modelos de árboles**. El MLP Básico alcanzó un AUC-ROC de 0.8197 y el MLP Profundo de 0.7705, quedando por debajo. Este resultado es esperable en datasets tabulares con variables estructuradas: los árboles capturan mejor las relaciones no lineales entre variables categóricas y numéricas.

Métricas adicionales — el efecto del desbalanceo:

Observación	Explicación
Accuracy baja (~70%)	Engañosa: un modelo que prediga siempre "no interesado" acertaría el 87% pero sería inútil.
Recall muy alto (~91%)	Los modelos detectan bien a casi todos los interesados.
Precision baja (~29%)	Muchos falsos positivos: se contacta a clientes que no comprarán. Es el precio a pagar por no perder oportunidades.
F1 moderado (~0.44)	Coherente con el trade-off Precision-Recall anterior.

Conclusión de negocio: el modelo prioriza **capturar todas las oportunidades** (Recall alto) aunque genere ciertos falsos positivos (Precision baja). Esta estrategia es la óptima cuando el coste de contactar a un cliente es bajo comparado con el valor de una póliza de vehículo.

 Ver curvas ROC comparadas: outputs/charts/roc_curves_final.png



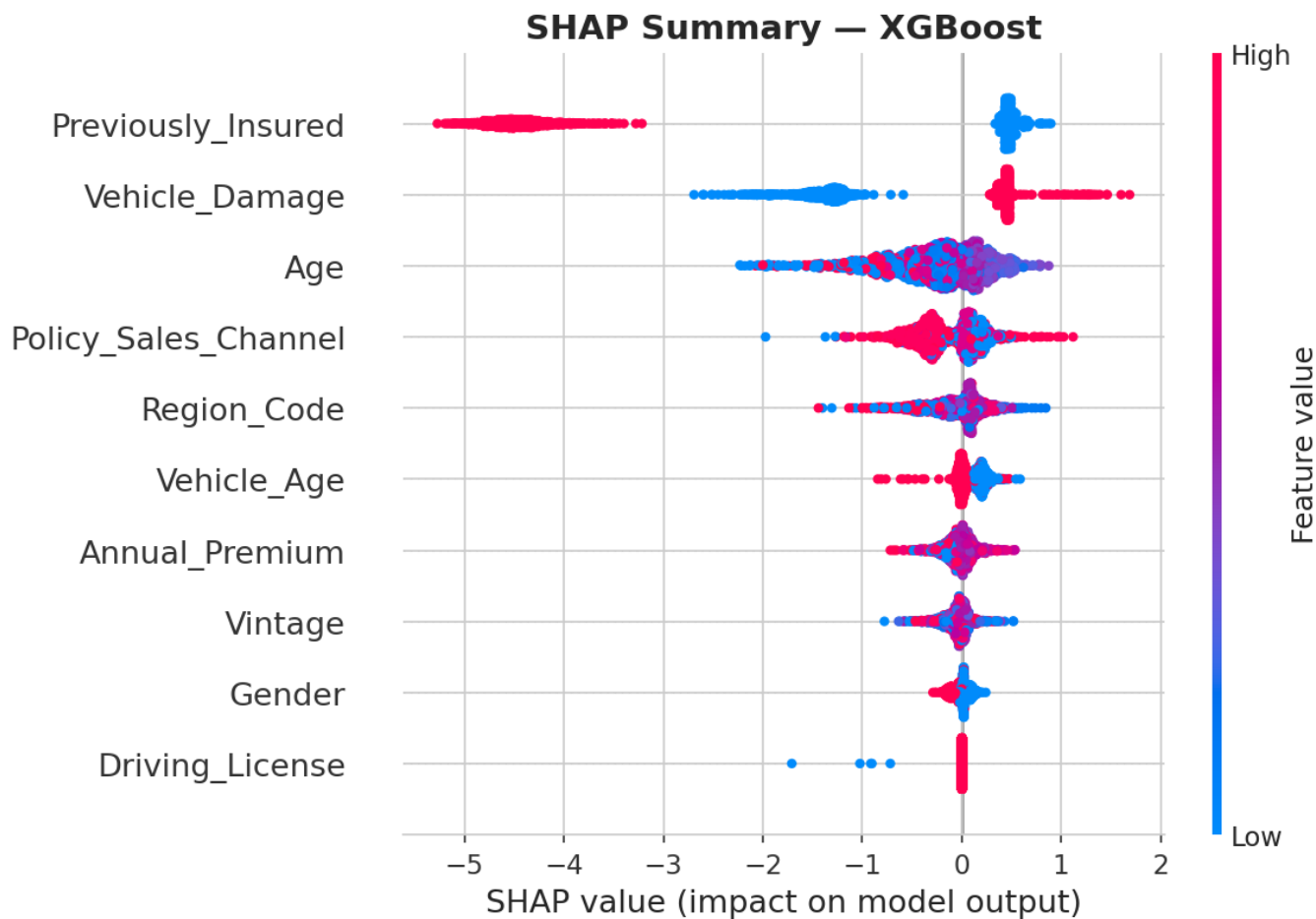
8. Interpretación de Resultados

8.1. Importancia de variables

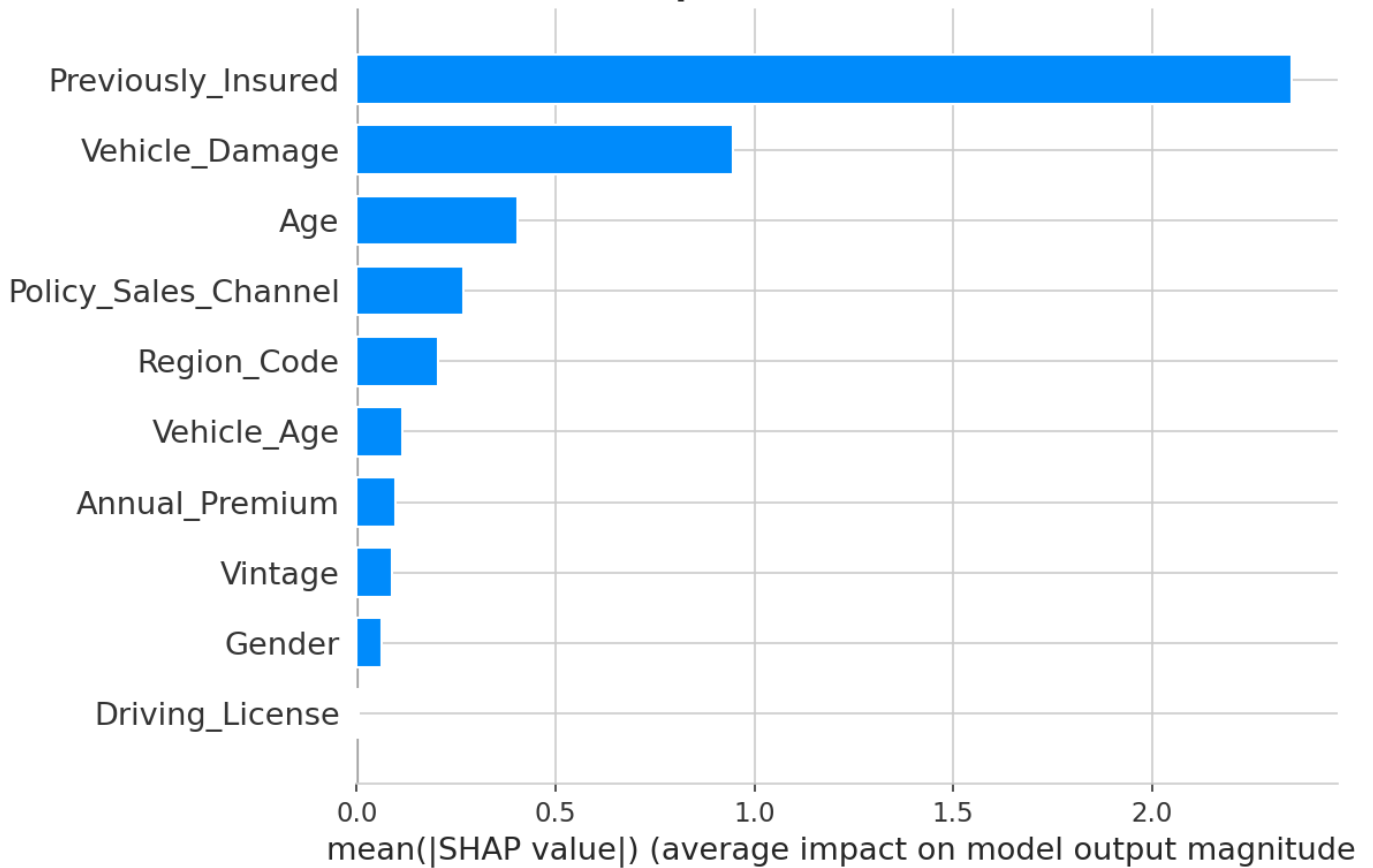
Análisis con **SHAP** y **Feature Importance** de los tres modelos de árbol:

Ranking	Variable	Interpretación de negocio
1	Previously_Insured	Cliente ya asegurado → casi nunca contrata
2	vehicle_Damage	Vehículo con daños → mayor propensión
3	Age	Clientes de mediana edad = mayor interés
4	Annual_Premium	Prima alta → más receptivo
5	vintage	Antigüedad moderada como señal

Ver gráficos: [shap_summary.png](#) · [shap_bar.png](#)



SHAP — Importancia media de variables



8.2. Tasa de conversión por segmentos

Segmento	Tasa de conversión
<code>Previously_Insured = 1</code>	~0.1%
<code>Previously_Insured = 0</code>	~22%
<code>Vehicle_Damage = 0</code>	~0.5%
<code>Vehicle_Damage = 1</code>	~23%

Insight clave: la combinación `Previously_Insured = 0` + `Vehicle_Damage = 1` concentra la mayor parte de la conversión real.

9. Recomendaciones de Negocio

- Dirigir campañas a clientes con `Previously_Insured = 0`.** Los ya asegurados no convierten (tasa ~0.1%). Excluirlos reduce el coste de campaña.
- Priorizar clientes con `Vehicle_Damage = 1`.** Su tasa de conversión es ~23%, frente a ~0.5% de los que no tienen daños.
- Segmentar por edad.** Los clientes entre 30 y 50 años tienen la mayor propensión.
- Optimizar el canal de contacto.** Analizar qué códigos de `Policy_Sales_Channel` convierten mejor para asignar presupuesto.
- Aplicar el modelo en producción.** Desplegar el mejor modelo (XGBoost o LightGBM) como servicio de scoring para priorizar leads en tiempo real.

10. Limitaciones y Trabajo Futuro

Limitaciones

- Fuerte desbalanceo inherente al problema (~12% positivos).
- Variable `Policy_Sales_Channel` es anónima → difícil interpretar canales.
- Sin datos temporales, no se puede analizar estacionalidad.

Trabajo futuro

- **Oversampling** con SMOTE para probar mejoras en F1.
 - **Hyperparameter tuning** con Optuna para exprimir XGBoost.
 - **Modelo de propensión con precio óptimo** para maximizar beneficio esperado.
 - **Despliegue con FastAPI** para servir predicciones en producción.
-

11. Conclusiones

Este proyecto demuestra que es posible **predecir con alta precisión** qué clientes tienen mayor probabilidad de contratar un seguro de vehículo, permitiendo a la compañía:

- **Reducir el coste de campañas** al no contactar clientes con tasa de conversión casi nula.
- **Aumentar la tasa de conversión** al priorizar los segmentos correctos.
- **Optimizar el canal de contacto** según el perfil del cliente.

Modelo ganador: LightGBM con un **AUC-ROC de 0.8574**, que supera a las arquitecturas de Deep Learning implementadas y confirma que los modelos basados en árboles son la opción más eficiente para este tipo de datos tabulares.

Impacto de negocio potencial: con una capacidad discriminativa del 85.7%, la compañía puede concentrar sus esfuerzos comerciales en el ~10% de clientes con mayor probabilidad predicha, donde se concentra la mayoría de las conversiones esperadas.

12. Anexos

-  Código: [notebooks/](#) y [scripts/](#)
-  Gráficos: [outputs/charts/](#)
-  Métricas: [outputs/metrics/](#)
-  Modelos entrenados: [models/trained/](#)